LangChain & LangGraph: Student Handout

Oct 4, 2026 · @Someone

Start here

In this lab you build a small AI travel agent, run it on test cases, and then grade how well it behaved. This handout explains every new word you will meet, in the order you meet it.

What is an AI agent? A program that decides for itself which step to take next, one step at a time, until a job is done. Our agent books travel: it can search for flights, book hotels, and check a calendar.

The two libraries, in one line each:

- **LangChain** gives the agent its tools: the actions it is allowed to take.
- **LangGraph** controls the flow: which step runs next, and when to stop.

One surprise before you start: there is no real AI model in this lab. Normally an AI model (like GPT or Claude) decides each step. Here, that decision-maker is replaced by a script: a Python dictionary called `SCENARIOS` that lists exactly what the agent will do. This keeps the lab free, gives the same results every run, and lets the lab plant specific mistakes on purpose. The graph, the tools, and all the grading code are real.

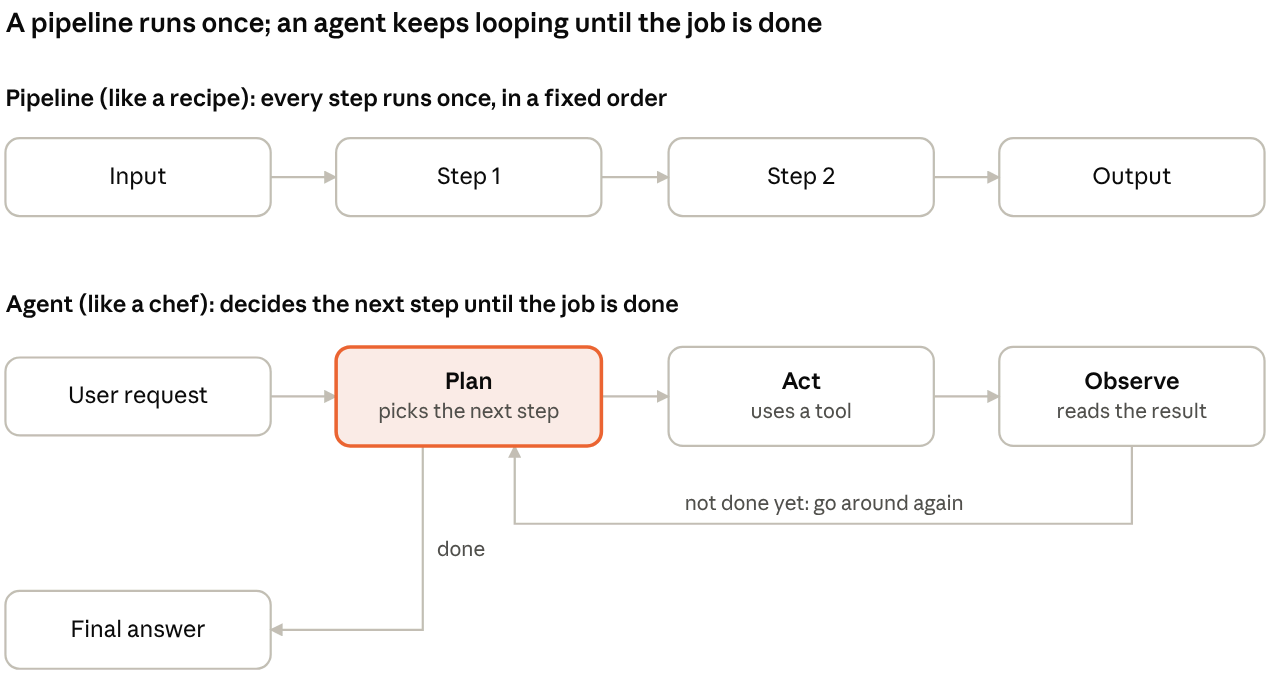
The lab in four stages:

1. Define the tools (Task 2)
2. Build the agent as a graph (Task 3)
3. Run 10 test cases and record what happened (Task 4)
4. Grade those records, layer by layer (Tasks 6 to 12)

What you need already: basic Python (dictionaries, lists, functions). Everything else is explained below.

What is an agent? A pipeline vs. a loop

A pipeline runs its steps once, in a fixed order. An agent keeps looping, and decides each next step as it goes.



pipeline vs. agent · a fixed line vs. a plan-act-observe loop

The highlighted **Plan** step is the decision point. Building this loop, and making every pass through it visible, is LangGraph's job in this lab.

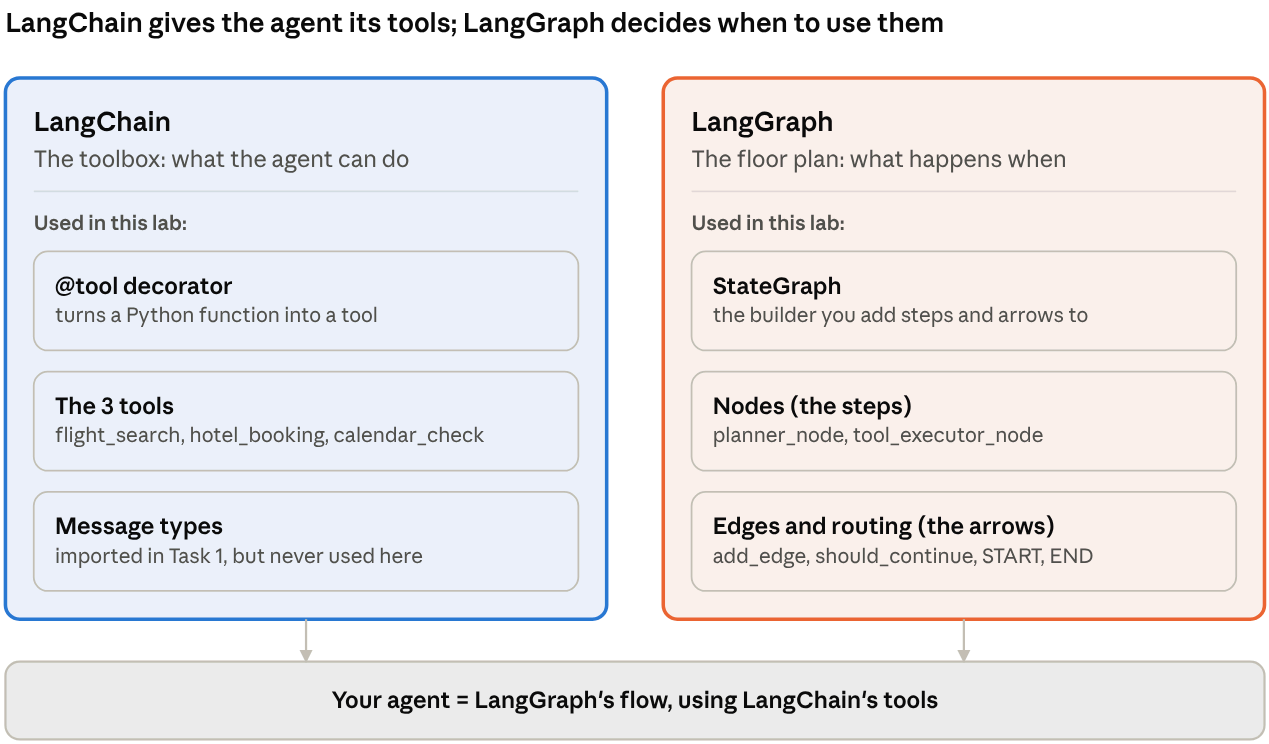
An analogy: a recipe says "do step 1, then step 2." A chef tastes the soup after each step and decides what to do next. An agent is the chef.

This plan-act-observe loop has a name you will see in the notebook: the **ReAct pattern** (short for Reason + Act).

- **Plan (reason):** decide what to do next.
- **Act:** use a tool.
- **Observe:** read what the tool sent back, then plan again.

LangChain vs. LangGraph

LangChain supplies the parts; LangGraph decides the order they are used in. In this lab, LangChain contributes only the `@tool` decorator and the message types. Everything about the flow is LangGraph.



LangChain vs. LangGraph · what each library contributes to this lab

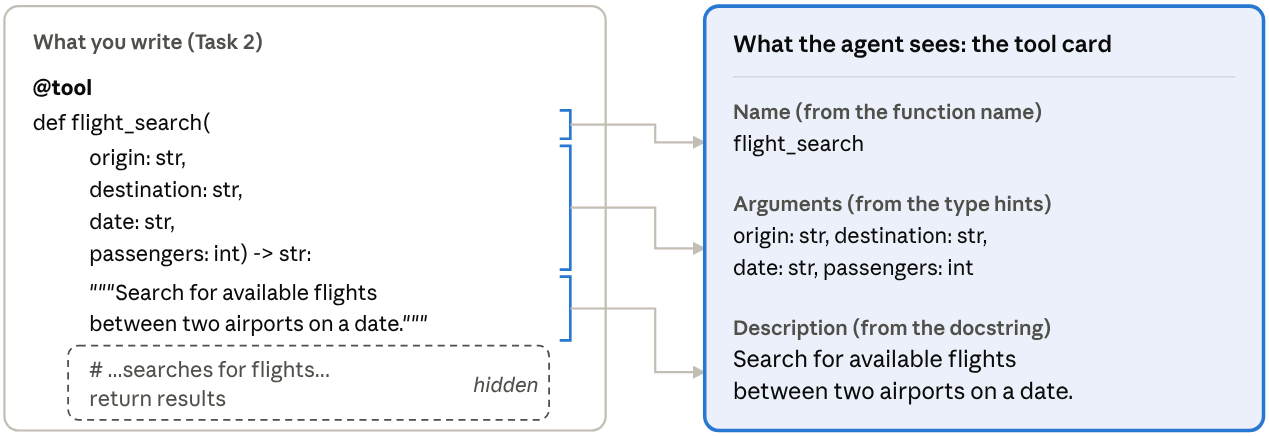
The later diagrams keep these colors (blue for LangChain, orange for LangGraph), so you can always tell which library a part comes from.

Do you need both? No. Each library works on its own. Many projects use them together, like this lab does. Outside this lab, LangChain also offers ready-made connections to AI models, documents, and databases. You won't need any of that today.

Tools and the @tool decorator (Task 2)

A **tool** is an ordinary Python function that the agent is allowed to call. The `@tool` decorator builds a "tool card" from that function, and the card is all the agent ever sees.

@tool turns a Python function into a tool card the agent can read



The code inside the function stays hidden. The agent only ever reads the card.

@tool · how a function becomes the tool card the agent reads

A **decorator** is the line starting with `@` just above a function. It adds extra behavior without changing the function's code. Here, `@tool` reads the function's name, its **type hints** (the `: str` and `: int` labels), and its **docstring** (the text inside `"""`).

Why it matters: a real AI model picks tools by reading these cards. A vague docstring or missing type hints leads to wrong tool calls, and wrong tool calls are exactly what this lab grades.

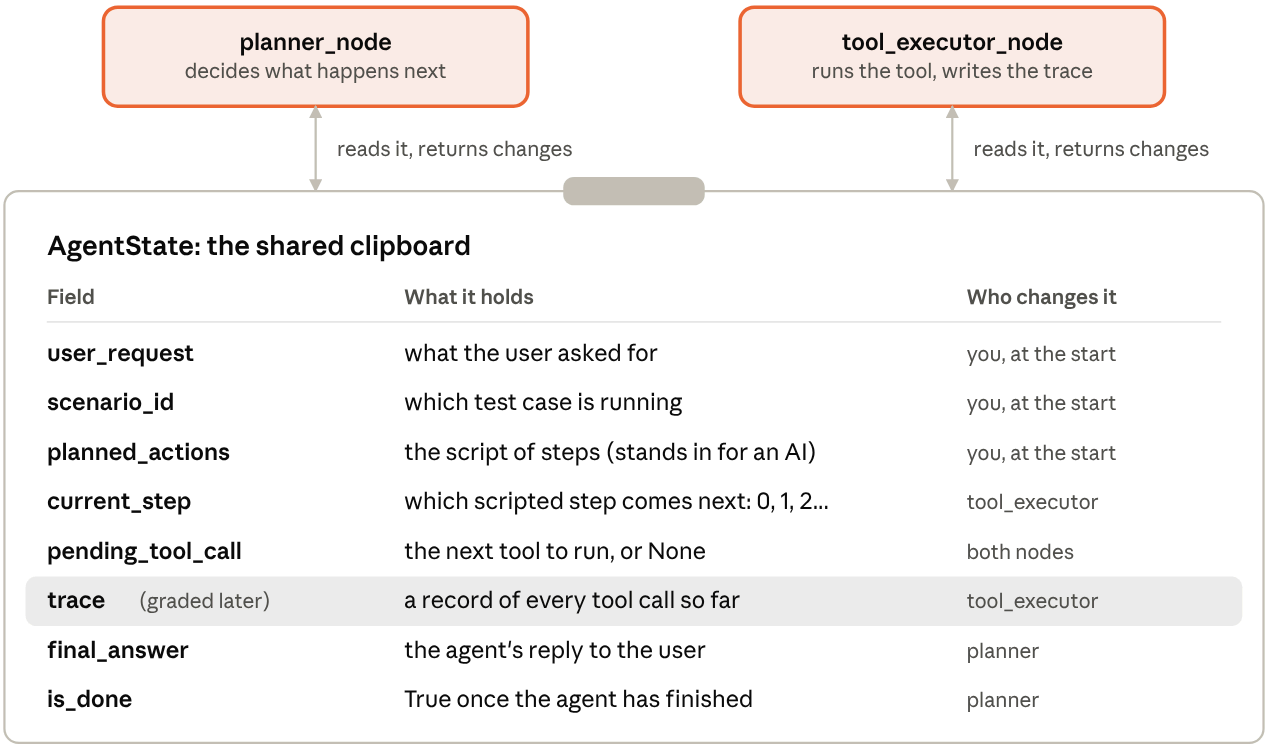
Three more things to know:

- The lab's tools are fake. They return made-up results instead of contacting real airlines or hotels. They still check their inputs and return an error for bad ones, such as a date written as "April 5".
- Every tool returns its result as a **JSON string**: text shaped like a Python dictionary, for example `{"status": "success", ...}`.
- To run a tool yourself, call `.invoke()` with a dictionary of inputs, for example `calendar_check.invoke({"user_id": "user_current", "date_range_start": "2026-03-27", "date_range_end": "2026-03-27"})`.

State: the agent's shared clipboard (Task 3, Step 1)

State is one dictionary that travels through the whole graph. Every step reads it, and every step can change parts of it. In this lab it is called `AgentState`.

State is one shared clipboard that every step reads and changes



AgentState · the 8 fields and which step changes each

How a step changes the state: a node never edits the clipboard directly. It returns a small dictionary holding only the fields it wants to change, and LangGraph copies those values onto the clipboard. For example, the planner returns `{"pending_tool_call": {...}, "is_done": False}`, and every other field stays as it was.

The field to watch is `trace`. Each time a tool runs, the executor adds one entry to it. When the run ends, `trace` holds the full history, and Tasks 6 to 12 grade that history.

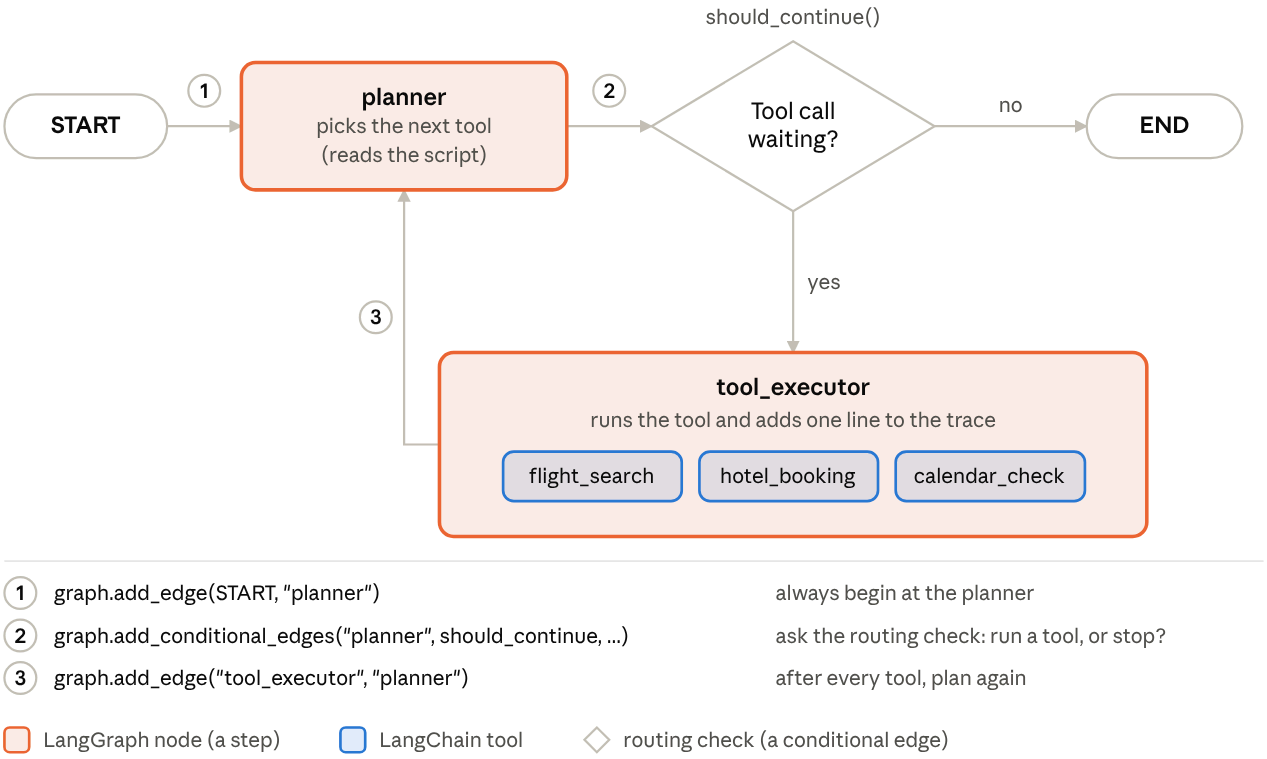
Two code words you will see here:

- `TypedDict` describes the shape of a dictionary: which keys it has, and what type each value is. LangGraph uses it to know what the clipboard holds.
- `Annotated` is imported but never used. In bigger projects it tells LangGraph to *add to* a field instead of replacing it. Here, the executor builds the new, longer `trace` list itself.

Nodes, edges, and routing: the lab's graph (Task 3)

A **graph** is a set of steps joined by arrows. The steps are called **nodes**, and the arrows are called **edges**. This lab's graph has just two nodes and one decision, and that is a complete agent.

The lab's agent: plan, run a tool, plan again, until done



the lab's graph · 2 nodes, 1 routing check, and the 3 lines of code that wire them

Follow the arrows: start at the planner, check whether a tool call is waiting, run it, then come back to the planner. The loop ends when the planner sets `is_done` to True and the routing check sends the agent to `END`.

A node is just a Python function. Every node has the same shape: `def my_node(state) -> dict:`. It receives the clipboard and returns the fields it changed. Once that clicks, LangGraph stops being mysterious.

Fixed vs. conditional edges. A fixed edge (1 and 3) always goes the same way. A conditional edge (2) asks a **routing function** every time. That one decision point is what makes this an agent instead of a pipeline.

The routing function in plain words. `should_continue` looks at the clipboard. If `is_done` is True, it returns `END`. If a tool call is waiting, it returns `"tool_executor"`. Otherwise it returns `END`.

Building the graph takes four moves:

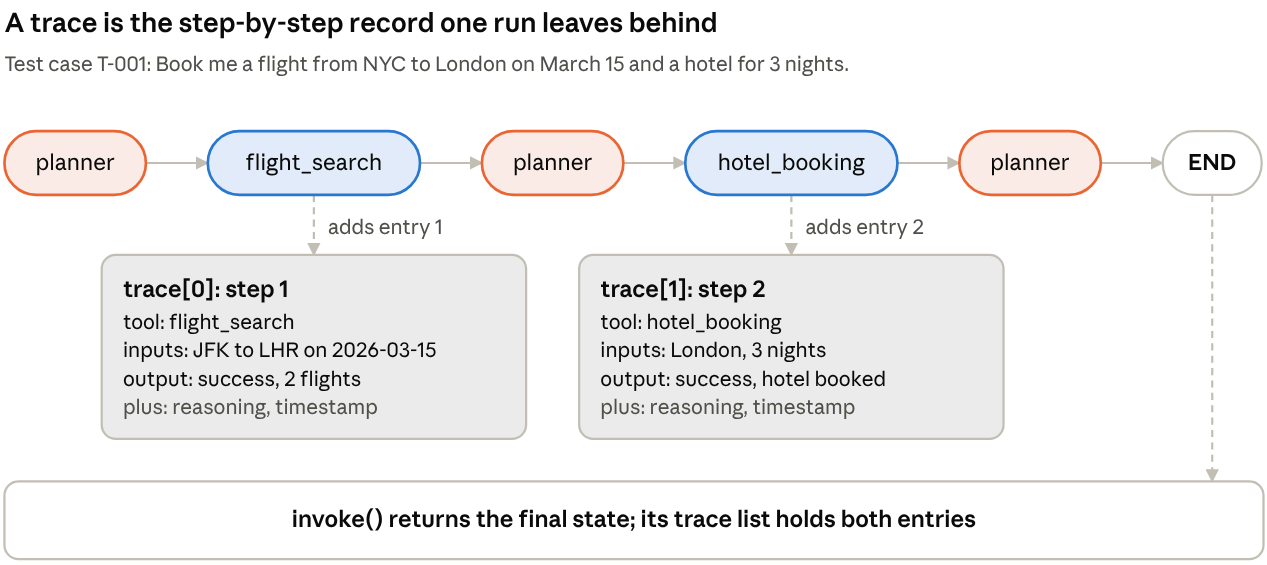
1. Create the builder: `graph = StateGraph(AgentState)`
2. Add the nodes: `graph.add_node("planner", planner_node)` and `graph.add_node("tool_executor", tool_executor_node)`
3. Add the edges: the three numbered lines in the diagram
4. Compile it: `agent = graph.compile()`. Compiling checks for mistakes, such as an arrow pointing at a node that doesn't exist, and returns an agent you can run.

Where would a real AI go? Only inside `planner_node`. Swap the script lookup for a call to an AI model, and every other part of the graph stays the same.

Optional: `print(agent.get_graph().draw_mermaid())` prints a text version of your compiled graph that you can paste into a Mermaid diagram viewer. Try it before class, since the output varies between library versions.

Running the agent: invoke() and traces (Task 4)

`agent.invoke(initial_state)` runs the whole graph from `START` to `END` and hands back the final clipboard. The most useful thing in it is the **trace**: one entry for every tool the agent called.



one run of T-001 · 5 visits to nodes, 2 trace entries

Each trace entry records six things: the step number, the tool, its inputs, its output, the reasoning, and a timestamp. Here is entry 1 from T-001, as Python stores it:

```
{"step": 1,
 "tool": "flight_search",
 "inputs": {"origin": "JFK", "destination": "LHR", "date": "2026-03-15",
"passengers": 1},
 "output": {"status": "success", "flights": [...], "currency": "USD"},
 "reasoning": "User wants a flight from NYC to London. Using JFK and LHR
airport codes.",
 "timestamp": "..."}
```

What `run_scenario` does, step by step:

1. Looks up a test case in `SCENARIOS`.
2. Fills a starting clipboard: the request, the scripted actions, `current_step = 0`, and an empty `trace`.
3. Calls `agent.invoke(...)`, which loops until `END`.
4. Attaches the **ground truth** (what the agent *should* have done), so the trace can be graded.

Words for this stage:

- **Scenario (test case):** one scripted situation, like T-001 or T-004.

- **Trajectory:** the whole path the agent took in one run. In this lab, it is the trace read from start to finish.
- **Ground truth:** the correct answer, written in advance: `ground_truth_tools`, `ground_truth_outcome`, and `expected_status`.

One quirk: the fake tools use Python's `hash()` to invent prices and flight numbers, and `hash()` gives different numbers in each new Python session. Prices can change between runs, and once in a while a hotel search under $150 returns `no_results` instead of `success`. Don't worry if your numbers differ a little from a classmate's.

Grading the agent (Tasks 5 to 12)

Once you have traces, the rest of the lab grades them in five layers. Each layer catches a mistake the earlier layers miss.

Layer	The question it asks	Task	Example it catches
1. Tool selection	Did the agent call the right tools?	6	T-002 searched flights to check a calendar
2. Input validation	Were the inputs well-formed and sensible?	7	T-003 sent the date "April 5" instead of "2026-04-05"
3. Trajectory score	How good was the whole run, on four measures?	8	T-005 scores below 1.0 despite a "success"
4. Root cause	Which step went wrong, and why?	10	T-006 said "free all day" despite a 2 PM meeting
5. Loop detection	Is the agent stuck repeating itself?	12	T-004 ran the same hotel search 5 times

Layer 1: precision and recall (Task 6)

Two numbers compare the tools the agent used with the tools it should have used.

- **Precision:** of the tools it called, what share were needed? Low precision means wasted work.
- **Recall:** of the tools it needed, what share did it call? Low recall means unfinished work.

```
\text{Precision} = \frac{\text{tools used that were needed}}{\text{tools used}} \qquad \text{Recall} = \frac{\text{tools used that were needed}}{\text{tools needed}}
```

Example: T-010 needed 3 tools but only called `flight_search`. Its precision is 1.00 (everything it did was right), but its recall is 0.33 (it did a third of the job).

Layer 2: schema vs. semantic errors (Task 7)

- **Schema error:** the input has the wrong format. Example: `date="April 5"` instead of `"2026-04-05"` (T-003).
- **Semantic error:** the format is fine, but the meaning makes no sense. Examples: checking out before checking in (T-009), or flying from NYC to NYC (T-002).

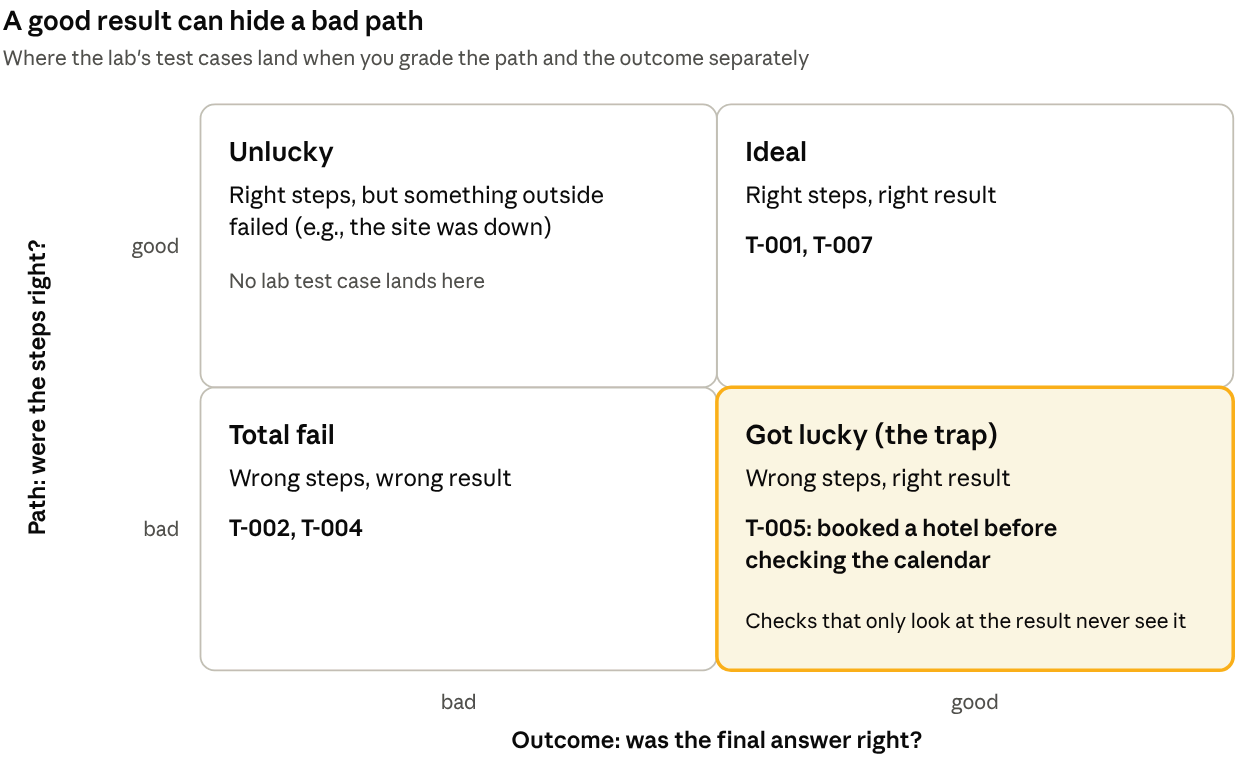
Layer 3: the trajectory score (Task 8)

Each run gets four scores from 0 to 1, then one weighted total.

Measure	Weight	How it is scored
Tool correctness	30%	Precision and recall combined into one number (the **F1 score**, which is high only when both are high)
Efficiency	20%	Ideal number of steps ÷ actual number of steps
Error handling	20%	Errors recovered from ÷ errors hit
Outcome quality	30%	Success = 1, partial = 0.5, failure = 0

In this lab, outcome quality comes from the `expected_status` label written into each scenario. A real system would need a person, or a grading model, to judge it.

The big lesson: path vs. outcome (Task 9)



path vs. outcome · the four quadrants from Task 9

T-005 booked a hotel before checking the calendar. The calendar happened to be clear, so the result looked fine. On a day with a conflict, the same habit would leave the user paying for a hotel they can't use. The notebook asks you to place T-006 and T-008 on this grid yourself.

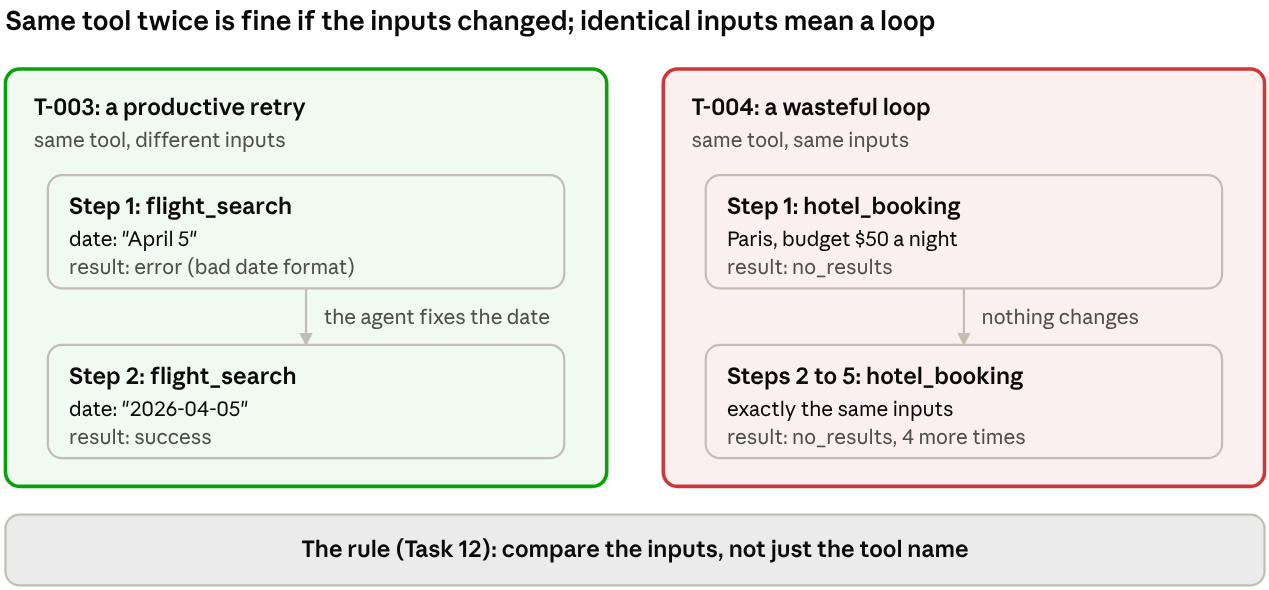
Layer 4: root causes (Task 10)

The analyzer labels each run with its most serious problem:

- **Output misinterpretation (critical):** right tools, valid inputs, correct data, wrong conclusion. T-006 read a 2 PM meeting and told the user they were free. Every other check passes, which makes this the most dangerous failure.
- **Wrong tool (high):** a needed tool was never called (T-002).
- **Looping (high):** the same call repeated with nothing changed (T-004).
- **Premature termination (high):** the agent stopped before finishing (T-010).
- **Unnecessary actions (medium):** extra tool calls nobody asked for (T-008).

Worth noticing: the misinterpretation check simply searches the final answer for the words "free" or "clear." Task 11's DEBUG-A shows the limit of rules like this. The agent booked the first flight instead of the cheapest, and no automated check notices.

Layer 5: loops vs. retries (Task 12)



loop vs. retry · T-003 compared with T-004

Calling the same tool twice is not a problem on its own. If the inputs changed, the agent learned from an error. If the inputs are identical, it is stuck, and every repeat costs time and money.

Glossary

Every term from the lab, in the order you meet it.

Term	Plain meaning	Where it appears
Agent	A program that picks its own next step until a job is done	The travel booking agent
LangChain	A library of building blocks for AI apps; here it supplies the tools	`langchain_core` (Tasks 1 and 2)
LangGraph	A library for building agents as graphs of steps	`langgraph.graph` (Task 3)

Term	Plain meaning	Where it appears
Tool	A Python function the agent is allowed to call	`flight_search`, `hotel_booking`, `calendar_check`
`@tool`	A decorator that turns a function into a tool card	Task 2
Decorator	A line starting with `@` that adds behavior to the function below it	`@tool`
Docstring	The text inside `"""` quotes; it becomes the tool's description	Each tool
Type hint	A label like `: str` or `: int` that says what kind of value an input is	Each tool's inputs
Schema	The list of a tool's inputs and their types	`t.args`
JSON string	Text formatted like a Python dictionary	What every tool returns
Message types	Typed chat messages: `HumanMessage`, `AIMessage`, `ToolMessage`	Imported in Task 1, never used
State	The shared dictionary every step reads and changes	`AgentState`
`TypedDict`	A way to describe a dictionary's keys and value types	`class AgentState(TypedDict)`
Graph	Steps (nodes) joined by arrows (edges)	The whole agent
`StateGraph`	The builder for a graph that passes state between steps	`StateGraph(AgentState)`
Node	A function that takes the state and returns the fields it changed	`planner_node`, `tool_executor_node`
Edge	A fixed arrow: after A, always go to B	`add_edge`

Term	Plain meaning	Where it appears
Conditional edge	An arrow chosen while the agent runs, by a routing function	`add_conditional_edges`
Routing function	Looks at the state and returns the name of the next node	`should_continue`
`START` / `END`	Built-in entry and exit points of every graph	Task 3
`compile()`	Checks the graph for mistakes and makes it runnable	`graph.compile()`
`invoke()`	Runs something: one tool, or the whole graph from `START` to `END`	`tool_fn.invoke(...)`, `agent.invoke(...)`
Planner	The node that decides the next step; normally an AI model, here a script	`planner_node`
Tool executor	The node that runs the chosen tool and records what happened	`tool_executor_node`
ReAct pattern	Reason, act, observe, repeat: the agent loop	The graph's loop
Scenario	One scripted test case	`SCENARIOS`: T-001 to T-010, DEBUG-A to DEBUG-C
Trace	The record of every tool call in one run	`state["trace"]`
Trajectory	The whole path of one run: the trace read start to finish	Task 8
Ground truth	The correct answer, written down in advance	`ground_truth_tools`, `ground_truth_outcome`
Precision	Of the tools called, the share that were needed	Task 6
Recall	Of the tools needed, the share that were called	Task 6

Term	Plain meaning	Where it appears
F1 score	One number that is high only when precision and recall are both high	Task 8
Root cause	The step where things first went wrong, and why	Task 10
Loop	The same tool called again with identical inputs	Task 12
Retry	The same tool called again with changed inputs	Task 12

Common confusions

- **"Where is the AI?"** There isn't one. The planner reads from `SCENARIOS`. If you swapped in a real AI model, only `planner_node` would change.
- **"Does `invoke()` mean two things?"** Yes. On a tool, `.invoke()` runs one function. On the compiled agent, `.invoke()` runs the whole loop from `START` to `END`.
- **"Why import message types that are never used?"** They show how a real agent talks to an AI model: the user's message, the model's reply, and each tool's result. This lab's script doesn't need them.
- **"The notebook says state is immutable."** That's a loose way to put it. More accurately: nodes return updates, and LangGraph copies them onto the state.
- **"Is `END` a node?"** It is a special built-in marker (its value is the text `"__end__"`). Routing to it tells LangGraph the run is over.

Check yourself

1. What is the difference between a node and an edge?
2. What would go wrong if `tool_executor_node` stopped writing to `trace`?
3. Why does the graph send the agent back to the planner after every tool?
4. If you replaced the script with a real AI model, which part of the code would change?
5. T-010's precision is 1.00, but its recall is 0.33. In plain words, what did the agent do?
6. Why is T-005 more dangerous than T-002, even though T-005's result looked fine?

Answers

1. A node is a step: a function that does some work. An edge is an arrow that says which step comes next.
2. The agent would still run, but Tasks 6 to 12 would have nothing to grade, because every layer reads the trace.
3. So the planner can see what the tool returned and decide what to do next. That loop is what makes it an agent.
4. Only `planner_node`. The tools, the graph, and the grading code stay the same.
5. Everything it did was correct, but it stopped after one of the three jobs it was given.
6. T-002 fails where everyone can see it, so someone fixes it. T-005 passes checks that only look at the result, so its risky habit stays hidden until it causes real harm.